CoReOS: A Conflict-Aware and Self-Healing Multi-Agent Operating System for Coordinated Resource Scheduling

[Author Name 1], [Author Name 2], [Author Name 3]
[Department of Computer Science and Engineering]
[University / Research Institution]
[City, State, Country]
[email1, email2, email3]@institution.edu

Abstract—Modern multi-tenant operating systems encounter severe cross-subsystem interference when specialized compute, memory, and storage resource-management policies operate in isolation. In uncoordinated systems, locally optimal scheduling decisions in one subsystem frequently trigger cascading performance pathologies in adjacent domains, such as memory thrashing, severe I/O seek starvation, and convoy stalls. This paper presents CoReOS, a conflict-aware, self-healing multi-agent operating system simulation framework designed for coordinated multi-resource arbitration. CoReOS models operating-system dynamics through a high-fidelity discrete-event engine orchestrating four CPU cores, a 16 GB physical memory space with swap paging, and dual-channel storage channels. Specialised domain agents—governing CPU dispatch, page reclamation, and storage queues—propose subsystem actions that are mediated by an adaptive coordination agent. This coordinator incorporates an I/O Congestion Shield, memory-CPU coupling constraints, and a calibrated machine-learning conflict prediction engine. Furthermore, a non-oscillating proportional self-healing engine dynamically detects throughput collapse, latency spikes, and thrashing, applying graduated corrective interventions. We conduct a rigorous empirical evaluation comprising 1,440 matched simulation trials across six heterogeneous workloads (steady_state, cpu_spike, memory_thrashing, io_bottleneck, mixed, and adversarial_stress) evaluated against seven competing architectures on 30 held-out pseudo-random seeds. Experimental results confirm that CoReOS ranks second overall with an aggregate composite score of 84.87 ± 1.35 points, outperforming conventional round-robin scheduling by $+10.58$ points ($t = 8.589, p = 4.20 \times 10^{-15}$, Cohen's $d = 0.640$) and the historical Complete CoReOS architecture by $+9.41$ points ($t = 9.452, p = 1.87 \times 10^{-17}$, $d = 0.704$). CoReOS achieves the highest overall task completion rate (95.86%) and slashes penalized turnaround time by 50.1% (40.79 vs. 81.73 ticks) while virtually eliminating thrashing (0.02 ticks). Crucially, CoReOS satisfies the strict ≤ 1.0 ms decision-overhead guardrail across 100% of evaluated trials (averaging 0.1517 ms). However, multi-criteria guardrail analysis exposes critical fairness bottlenecks in I/O-intensive workloads (Jain's index of 0.3934), demonstrating that overhead compliance does not guarantee universal guardrail satisfaction and revealing survivorship bias in unpenalized conventional baselines.

Index Terms—Autonomic computing, discrete-event simulation, multi-agent systems, multi-objective optimization, operating system scheduling, resource arbitration, self-healing systems.

I. Introduction

Multi-resource contention represents one of the most persistent bottlenecks in modern systems architecture. Contemporary computing workloads—spanning distributed microservices, parallel training pipelines, and interactive web services—place simultaneous, non-stationary demands across central processing units (CPU), physical random-access memory (RAM), virtual paging swap space, and non-volatile block storage subsystems [1], [2]. In standard Unix-like operating-system kernels, resource management has historically evolved as a set of decoupled, specialized subsystems: CPU threads are scheduled via proportional-share or multi-level feedback heuristics such as the Completely Fair Scheduler (CFS); memory frames are managed through asynchronous page replacement daemons (*kswapd*); and block storage requests are queued via seek-minimizing elevators or dispatch deadlines [3], [4].

While localized subsystem decoupling simplifies kernel abstraction layers, it inherently assumes that resource contention operates orthogonally across physical domains. In practice, however, hardware subsystems exhibit intense, non-linear cross-resource coupling [5]–[7]. For example, when an aggressive CPU scheduling policy deploys Shortest Remaining Time First (SRTF) or fine-grained Round Robin to accelerate task completions, it rapidly switches thread execution contexts. This churn forces uncoordinated memory allocators to rapidly map working sets into physical memory frames. If aggregate memory demand exceeds physical RAM capacity, the paging subsystem initiates emergency page swapping. The resulting influx of page-in and page-out requests saturates the underlying storage controller, producing disk queue starvation and inflating storage wait times. Consequently, tasks executing on the CPU stall waiting for synchronous I/O completion, triggering convoy delays, throughput collapse, and severe memory thrashing [8], [9].

To overcome these structural limitations, recent systems research has explored two distinct paradigms: (i) centralized heuristic cluster and kernel schedulers that predict in-

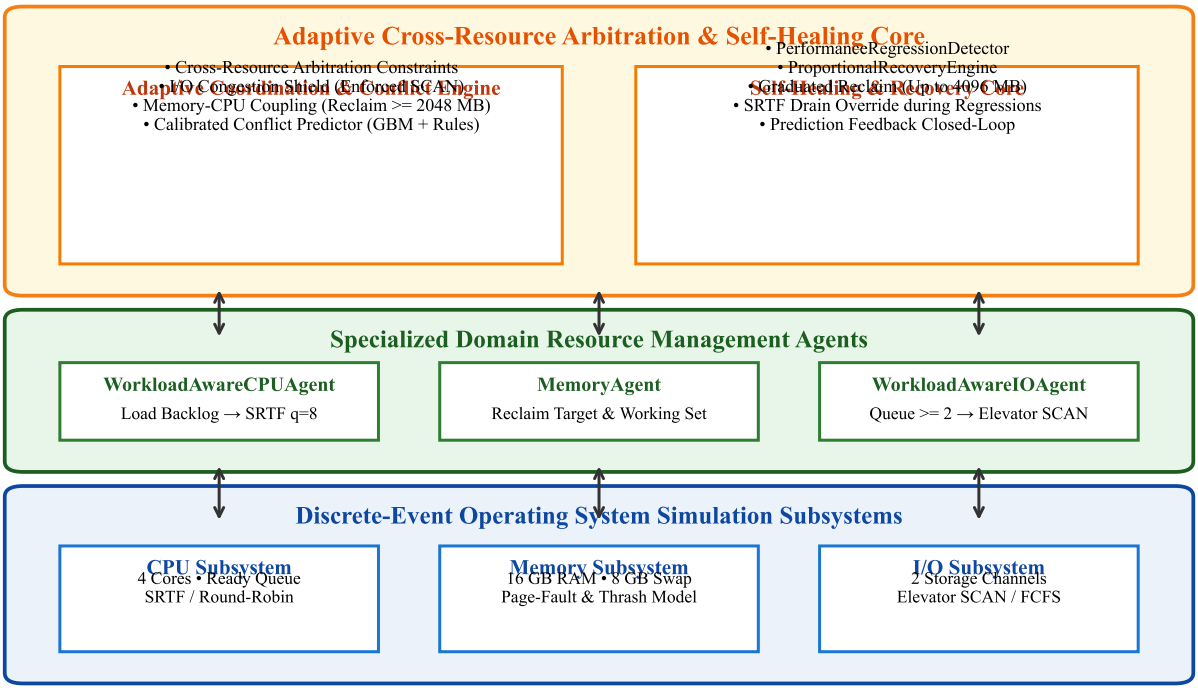


Fig. 1. Architectural overview of the CoReOS multi-agent operating system. Specialized domain agents monitor discrete-event subsystem states (CPU, memory, storage) and formulate independent action proposals. The Adaptive Coordination Agent applies deterministic safety constraints, cross-resource arbitration rules, and machine-learning conflict prediction. A graduated Proportional Recovery Engine tracks regression telemetry and deploys closed-loop self-healing overrides.

terference patterns (e.g., Paragon [5], Heracles [6], Bubble-Up [7]); and (ii) reinforcement learning (RL) schedulers that train neural policies to make black-box allocation decisions (e.g., DeepRM [10], Decima [11]). Simultaneously, the rise of autonomous agent architectures has inspired the concept of "AI Operating Systems" (e.g., AIOS [12], AgentRM [13], AutoGen [14]), which structure software agents as applications coordinated by an orchestration kernel.

However, existing multi-agent operating systems primarily focus on coordinating Large Language Model (LLM) API calls, tool invocations, and context window tokens [12]. They do not address the foundational operating-system problem: how specialized, autonomous control agents can dynamically negotiate low-level CPU slices, page reclamation thresholds, and disk head scheduling without destabilizing the system. Conversely, black-box deep reinforcement learning approaches frequently suffer from severe sample inefficiency, unpredictable tail latency, and catastrophic policy degradation under unseen out-of-distribution workloads [15], [16].

To address this challenge, we introduce CoReOS (Conflict-Aware, Resource-Arbitrated Operating System), a specialized multi-agent operating system simulation framework designed for transparent, coordinated resource arbitration and closed-loop self-healing. CoReOS models system dynamics via a discrete-event execution engine that faithfully models CPU cores, physical memory allocation, page-fault penalties, disk seek geometry, and storage channels. Within CoReOS, specialized domain agents formulate resource proposals based on localized telemetry. These proposals are mediated by an Adaptive Coordination Agent that enforces explicit cross-resource coupling invariants, an I/O Congestion Shield, and a hybrid machine-learning conflict prediction engine. Furthermore, CoReOS integrates a non-oscillating Proportional Recovery Engine that detects performance regressions and applies graduated interventions to restore system equilibrium.

A. Research Questions

This investigation is structured around four primary research questions:

- RQ1 (Arbitration Efficacy): Can explicit cross-resource arbitration rules and conflict prediction prevent multi-subsystem interference and improve aggregate system throughput, completion rate, and turnaround latency compared to uncoordinated independent agents and conventional round-robin scheduling?
- RQ2 (Self-Healing Robustness): Does graduated, progress-aware proportional self-healing mitigate catastrophic thrashing and queue collapse under se-

vere adversarial stress without introducing oscillatory policy thrashing?
- RQ3 (Computational Feasibility): Can multi-agent arbitration and machine-learning conflict prediction execute within an operational decision overhead ceiling of $\leq 1.0\,\text{ms}$ per step in a discrete-event environment?
- RQ4 (Multi-Objective Fairness and Survivorship Bias): How does multi-resource arbitration affect process fairness across heterogeneous workloads, and does conventional unpenalized metric calculation mask severe survivorship bias in starved baselines?

B. Research Contributions

This paper presents four verified research contributions:

1) System Architecture and Discrete-Event Model: We design and implement the CoReOS framework, coupling a multi-core, paged-memory, and elevator-storage discrete-event simulation engine with specialized resource agents, deterministic safety constraints, and an explainable coordination core.
2) Conflict-Aware Cross-Resource Arbitration: We formulate and implement an adaptive arbitration mechanism incorporating an I/O Congestion Shield and Memory-CPU coupling rules, complemented by a calibrated Gradient Boosting conflict predictor that detects cascading multi-resource saturation before performance collapses.
3) Graduated Proportional Self-Healing: We develop a closed-loop performance regression detector and non-oscillating recovery engine that applies progress-aware SRTF queue draining and deep memory reclamation under memory overcommit, accompanied by an empirical feedback loop.
4) Comprehensive Empirical Validation and Bias Analysis: Across 1,440 canonical matched simulation trials spanning six diverse workloads and eight candidate architectures, we demonstrate that CoReOS ranks second overall (84.87 ± 1.35 composite score), outperforming conventional scheduling by $+10.58$ points ($p < 10^{-14}$) with 100% decision-overhead compliance ($0.1517\,\text{ms}$). Furthermore, we provide a mathematical analysis of survivorship bias in completed-job fairness metrics, identifying critical fairness bottlenecks under I/O-intensive workloads.

C. Paper Organization

The remainder of this paper is organized as follows. Section II reviews related literature across operating-system scheduling, multi-agent coordination, autonomic computing, and fairness. Section III establishes the formal discrete-event model, process lifecycle, and multi-objective optimization problem. Section IV details the architecture, algorithms, and complexity of CoReOS. Section V outlines the benchmark environment, workload taxonomy, candidate configurations, and statistical methods. Section VI presents the empirical results, guardrail compliance audits, and survivorship bias analysis. Section VII addresses threats to validity and limitations. Section VIII concludes and outlines future research directions.

II. Related Work and Background

A. Operating-System Scheduling and Resource Management

Classical operating-system CPU scheduling has long relied on queue-based heuristics designed to balance interactive latency and computational throughput [1]. The Shortest Remaining Time First (SRTF) discipline provides mathematically provable minimality in average turnaround time for single-resource systems; however, it is notoriously vulnerable to starvation for compute-intensive tasks and requires a priori knowledge of burst durations [2]. General-purpose kernels such as Linux have adopted proportional-share policies, epitomized by the Completely Fair Scheduler (CFS), which tracks per-task virtual runtime (*vruntime*) via red-black trees to guarantee fair allocation of processor cycles [3]. In storage architectures, disk scheduling algorithms have progressed from First-Come, First-Served (FCFS) to Shortest Seek Time First (SSTF), SCAN (elevator), and C-LOOK to amortize mechanical seek overheads [1]. In memory management, page replacement algorithms such as Least Recently Used (LRU), CLOCK, and working-set estimators endeavor to maintain physical memory headroom [2]. Nevertheless, conventional kernels treat these resource subsystems as independent silos, optimizing individual allocation queues without cross-subsystem visibility.

B. Cross-Subsystem Contention and Multi-Resource Interference

Hardware resource sharing in modern multicore and virtualized environments induces severe performance interference. Mars et al. [7] introduced Bubble-Up, quantifying how shared last-level cache (LLC) and memory bus contention degrade latency-critical applications in warehouse-scale clusters. Delimitrou and Kozyrakis developed Paragon [5] and Quasar [17], utilizing collaborative filtering and classification to predict application sensitivity to memory bandwidth, cache capacity, and network contention. Lo et al. [6] presented Heracles, a sub-millisecond hardware-software feedback controller that dynamically partitions CPU cores, memory bandwidth, and power budgets to maintain latency service-level objectives (SLOs) while co-locating batch workloads. Similarly, Kasture and Sanchez [9] demonstrated that uncoordinated memory reclamation triggers severe cache pollution and disk bandwidth collapse. While these datacenter-level systems effectively regulate hardware allocations across virtual machines, they operate above the OS abstraction layer, treating kernel scheduling policies as static black boxes.

C. Multi-Agent Coordination and Resource Arbitration

Multi-agent systems (MAS) provide a foundational framework for distributed decision-making where autonomous entities possess localized information and distinct objectives [18], [19]. Coordination in multi-agent environments typically requires establishing explicit communication protocols, social laws, or centralized arbitration mechanisms to prevent destructive resource conflict [20], [21]. In computational resource allocation, multi-agent frameworks have been applied to microgrid power routing, cellular base-station spectrum allocation, and cloud VM placement [16]. However, uncoordinated independent agents frequently suffer from the "moving target" problem, where simultaneous policy updates induce non-stationarity, leading to destructive oscillation and system-wide performance degradation [20]. CoReOS adapts the multi-agent paradigm to internal operating-system scheduling by structuring domain agents as proposal generators, mediated by a centralized, conflict-aware coordination agent.

D. Autonomic Computing and Self-Healing Systems

The vision of autonomic computing, articulated by Kephart and Chess [22], advocates for computing systems that self-configure, self-optimize, self-protect, and self-heal via continuous Monitor-Analyze-Plan-Execute (MAPE-K) loops. In systems software, fault recovery has focused on rapid state restoration. Candea et al. [23] pioneered microrebooting, demonstrating that surgically restarting fine-grained software components yields order-of-magnitude reductions in downtime compared to node-level reboots. Qin et al. [24] introduced Rx, which treats software bugs like environmental allergies by re-executing failing processes within modified memory and scheduling environments. Kramer and Magee [25] established an architectural framework for self-managed systems separating component-level control from high-level goal management. CoReOS adopts this graduated recovery philosophy, deploying a Proportional Recovery Engine that applies progress-aware queue draining and deep reclamation to recover from performance collapse without full process restarts.

E. Reinforcement Learning for Scheduling

The application of reinforcement learning to scheduling has gained substantial momentum [15]. Mao et al. [10] presented DeepRM, utilizing policy gradient algorithms to translate resource packing into visual state representations for multi-resource cluster allocation. In follow-up work, Mao et al. [11] introduced Decima, employing graph neural networks (GNNs) to learn scheduling policies for complex directed acyclic graph (DAG) dataflow jobs. While deep RL policies can discover non-intuitive scheduling heuristics, they exhibit fundamental vulnerabilities in operating-system environments: their inference latencies frequently exceed the millisecond kernel boundary, and their tabular or neural representations struggle to generalize when confronted with unprecedented workload stress or catastrophic thrashing.

F. Fairness Metrics and Multi-Objective Evaluation

Measuring fairness in shared computing resources requires robust mathematical foundations. In their seminal work, Jain, Chiu, and Hawe [26] formulated the Jain Fairness Index, an axiomatic metric bounded in $[1/n, 1.0]$ that evaluates whether shared allocations deviate from equitable distribution. Lan et al. [27] extended this theory, establishing axiomatic constraints for multi-resource fairness. In multi-objective systems evaluation, Deb [28] and Marler and Arora [29] demonstrated that optimizing competing criteria (e.g., maximizing throughput while minimizing turnaround time and overhead) requires Pareto analysis or carefully structured composite utility functions. Crucially, as we analyze in Section VI, evaluating fairness strictly over completed jobs introduces severe survivorship bias when schedulers starve tasks.

G. Research Gap and Positioning of CoReOS

Table I summarizes the architectural positioning of CoReOS relative to prior art. Existing research either focuses on macroscopic cloud datacenter co-location (e.g., Paragon, Heracles), post-crash software recovery (e.g., Rx), black-box neural cluster schedulers (e.g., DeepRM, Decima), or high-level LLM agent orchestration (e.g., AIOS, AgentRM). A critical research gap exists: there is no transparent, multi-agent operating-system framework that unifies discrete-event multi-resource execution, explicit cross-subsystem conflict prediction, explainable arbitration, and closed-loop self-healing within strict millisecond decision boundaries. CoReOS directly addresses this foundational void.

III. Problem Formulation and Preliminaries

A. Simulated System State and Subsystems

We formulate the multi-resource operating-system simulation as a discrete-event dynamical system operating over discrete time ticks $t \in \mathbb{R}^+$. The physical computing environment comprises three coupled hardware subsystems:

1) CPU Subsystem ($\mathcal{C}$): Consisting of $N_c = 4$ identical processor cores. At any time t, each core $c \in \{1, \ldots, N_c\}$ is either idle or executing an assigned process $p \in \mathcal{P}$. Process switching incurs a deterministic context-switch penalty $\delta_{cs} = 0.5$ ticks. The ready queue is denoted $\mathcal{Q}_{ready}$.
2) Memory Subsystem ($\mathcal{M}$): Consisting of physical random-access memory with capacity $M_{phys} = 16,384$ MB and virtual swap space capacity $M_{swap} = 8,192$ MB. Physical frame allocation is trackable per process: $m_p(t)$ denotes process p's physical footprint. Memory utilization ratio is defined as:

$$\mu_m(t) = \frac{\sum_{p \in \mathcal{P}_{active}} m_p(t)}{M_{phys}} \in [0, 1]. \tag{1}$$

Memory pressure is triggered when $\mu_m(t) \geq \theta_{press} = 0.85$. Page faults incur a latency cost $\delta_{pf} = 5.0$ ticks. Severe memory thrashing occurs when $\mu_m(t) \geq 0.92$ and active swap utilization exceeds 5% of swap capacity:

$$\text{Thrashing}(t) \iff \mu_m(t) \geq 0.92 \wedge \sum_{p \in \mathcal{P}} m_p^{swap}(t) \geq 0.05 \cdot \tag{2}$$

3) I/O Storage Subsystem ($\mathcal{I}$): Consisting of $N_{io} = 2$ independent storage channels servicing a storage request queue $\mathcal{Q}_{io}$ with capacity $K_{io} = 50$. Each request specifies an I/O burst duration d_{io} and a target disk cylinder $\ell \in [0, 100]$. The current disk head position is $h(t) \in [0, 100]$ moving with direction $dir(t) \in \{-1, +1\}$. Channel saturation is defined as $\sigma_{io}(t) = |\mathcal{Q}_{io}(t)|/K_{io}$.

B. Process Lifecycle and Execution Model

A workload $\mathcal{W}$ consists of N_p processes submitted over time: $\mathcal{W} = \{p_1, p_2, \ldots, p_{N_p}\}$. Each process p_i is characterized by a tuple:

$$p_i = \langle a_i, B_i, M_i, \rho_i, I_i \rangle, \tag{3}$$

where $a_i \geq 0$ is the arrival tick, $B_i > 0$ is total CPU burst demand, $M_i > 0$ is physical memory demand, $\rho_i \in \{1, \ldots, 5\}$ is scheduling priority, and I_i is a sequence of I/O request events.

At any tick t, a process resides in exactly one state $s_i(t) \in \mathcal{S}$:

$$\mathcal{S} = \{\text{PENDING}, \text{WAITING_MEM}, \text{READY}, \text{RUNNING}, \text{WAITING_IO}, \text{TERMINATED}\}. \tag{4}$$

The state transition dynamics are governed as follows:

- Admission: When $t \geq a_i$, process p_i transitions from PENDING. If $M_{phys} - \sum m_p(t) \geq M_i$, memory is allocated and $p_i \rightarrow$ READY. Otherwise, $p_i \rightarrow$ WAITING_MEM.
- Execution: From $\mathcal{Q}_{ready}$, a scheduler assigns $p_i \rightarrow$ RUNNING on core c. Execution advances by effective time $\Delta t/\omega_{slow}$, where $\omega_{slow} \geq 1.0$ represents thrashing slowdown.
- I/O Interruption: If p_i triggers an I/O burst, it yields core c and transitions to WAITING_IO, entering $\mathcal{Q}_{io}$.
- Completion: When remaining burst $r_i(t) \leq 0$, $p_i \rightarrow$ TERMINATED, recording completion tick $C_i = t$.

C. Multi-Objective Evaluation and Survivorship Bias

For each completed process p_i, turnaround time is $T_i = C_i - a_i$, and waiting time is $W_i = T_i - B_i$. System throughput is:

$$\Theta(t) = \frac{|\mathcal{P}_{comp}(t)|}{t} \quad [\text{processes/tick}]. \tag{5}$$

The completion rate across total submitted processes N_p is:

$$CR = \frac{|\mathcal{P}_{comp}|}{N_p} \times 100\%. \tag{6}$$

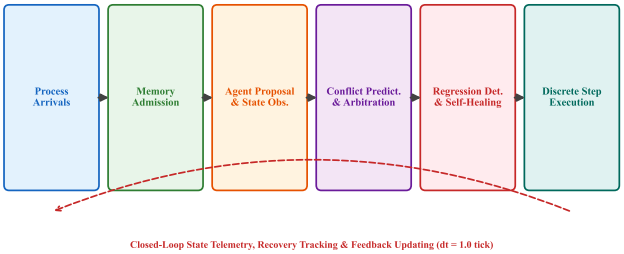


Fig. 2. End-to-end discrete-event simulation and arbitration workflow. At each tick, localized agent proposals are checked against deterministic safety invariants, cross-resource arbitration rules, and machine-learning conflict models before the simulation engine advances state.

Survivorship Bias in Completed-Job Metrics: Conventional benchmarks compute mean turnaround time $\bar{T}_{comp}$ strictly over completed processes:

$$\bar{T}_{comp} = \frac{1}{|\mathcal{P}_{comp}|} \sum_{p_i \in \mathcal{P}_{comp}} T_i. \tag{7}$$

When a candidate system starves long-running tasks under adverse load, only short, early-arriving tasks complete. Consequently, $\bar{T}_{comp}$ artificially drops, rewarding schedulers that fail under load. To eliminate this survivorship bias, CoReOS defines the diagnostic Penalized Turnaround Time (T_{pen}), charging an evaluation cutoff penalty $\Pi = 300.0$ ticks for every unfinished process:

$$T_{pen} = \frac{1}{N_p} \left[\sum_{p_i \in \mathcal{P}_{comp}} T_i + (N_p - |\mathcal{P}_{comp}|) \cdot \Pi \right]. \tag{8}$$

Jain's Fairness Index: Fairness is computed over completed process turnaround times via Jain's formulation [26]:

$$J(\mathbf{T}) = \frac{\left(\sum_{i=1}^{n} T_i\right)^2}{n \sum_{i=1}^{n} T_i^2} \in \left[\frac{1}{n}, 1.0\right]. \tag{9}$$

Composite Benchmark Score: To synthesize multi-resource trade-offs into a standardized competition metric ($S \in [0, 100]$), the canonical benchmark specifies:

$$\begin{aligned} S = 100 \times \Bigg[& 0.30 \left(\frac{CR}{100}\right) + 0.25 \min\left(1.0, \frac{\Theta}{0.25}\right) \\ & + 0.20 \max\left(0, 1.0 - \frac{T_{pen} - 5.0}{295.0}\right) \\ & + 0.15 \max\left(0, 1.0 - \frac{\text{ThrashingTicks}}{150.0}\right) \\ & + 0.10 \,\text{clamp}(J(\mathbf{T}), 0, 1) \Bigg]. \end{aligned} \tag{10}$$

IV. Proposed System: CoReOS

A. System Architecture and Overview

The CoReOS architecture (Fig. 1) establishes a multi-tiered control hierarchy comprising three decoupled functional layers:

1) Subsystem Simulation Layer: The underlying discrete-event engine modeling physical hardware queues, caches, and registers.
2) Domain Agent Layer: Three specialized agents—WorkloadAwareCPUAgent, MemoryAgent, and WorkloadAwareIOAgent—observing localized state vectors and proposing domain actions.
3) Arbitration and Self-Healing Layer: The AdaptiveCoordinationAgent, CalibratedConflictEngine, PerformanceRegressionDetector, and ProportionalRecoveryEngine, which collectively enforce cross-resource consistency and system stabilization.

B. Specialized Domain Resource Agents

Each agent observes the global system state $\mathbf{x}(t)$ and formulates a structured action proposal:

- WorkloadAwareCPUAgent: Observes ready queue depth $|\mathcal{Q}_{ready}|$, waiting memory count, CPU utilization, and total active load $L_{act} = |\mathcal{Q}_{ready}| + |\mathcal{Q}_{mem}| + |\mathcal{Q}_{io}|$. To resolve convoy stalls under compute bursts, the agent implements progress-aware detection:

$$\text{CPU Policy} = \begin{cases} \text{SRTF } (q = 8.0), & \text{if } |\mathcal{Q}_{ready}| \geq 3 \vee (L_{act} \geq 4 \wedge |\mathcal{Q}_{ready}| \geq 2) \\ & \quad \vee \text{ Utilization} \geq 85\%, \text{ non-I/O;} \\ \text{RR } (q = 6.0), & \text{if } L_{act} > 2 \vee |\mathcal{Q}_{ready}| > 1; \\ \text{RR } (q = 4.0), & \text{otherwise (light load).} \end{cases} \tag{11}$$

- MemoryAgent: Observes physical memory utilization $\mu_m(t)$, swap usage, and page fault rates. When $\mu_m(t) \geq 0.85$, it proposes an emergency reclamation target $R_{target} = 2,048\,\text{MB}$. Under moderate load ($\mu_m \geq 0.60$), it proposes $R_{target} = 1,024\,\text{MB}$.
- WorkloadAwareIOAgent: Observes storage queue backlog $|\mathcal{Q}_{io}|$ and channel saturation σ_{io}. To eliminate the catastrophic cylinder starvation inherent to SSTF, it mandates elevator SCAN whenever queue backlog is non-trivial:

$$\text{I/O Policy} = \begin{cases} \text{SCAN}, & \text{if } |\mathcal{Q}_{io}| \geq 2 \vee \sigma_{io}(t) \geq 30\%; \\ \text{FCFS}, & \text{otherwise (minimal queue).} \end{cases} \tag{12}$$

C. Cross-Resource Arbitration and Conflict Detection

When agents propose conflicting actions, the AdaptiveCoordinationAgent applies deterministic arbitration rules and statistical conflict prediction (Algorithm 1):

1) I/O Congestion Shield: If $|\mathcal{Q}_{io}| \geq 2$ or saturation $\geq 30\%$, elevator SCAN is strictly enforced, overriding any greedy agent request.
2) Memory-CPU Coupling Constraint: Under memory pressure ($\mu_m \geq 0.80$, thrashing, or waiting processes), CPU quantum is clamped to $q \geq 8.0$ ticks to amortize context switching and prevent page thrashing, while memory reclamation is elevated to $\geq 2,048\,\text{MB}$.
3) Preservation of SRTF Progress: If memory pressure is elevated but the CPU agent has deployed SRTF to clear compute backlog, SRTF is strictly preserved rather than forced into round-robin, preventing pipeline stalling.

Calibrated Conflict Engine: A hybrid prediction engine integrates deterministic expert cascade rules with an ensemble Gradient Boosting Classifier (50 estimators, max depth 3) trained on multi-resource dynamics. The model evaluates a 10-dimensional feature vector:

$$\mathbf{f}(t) = \langle \mu_c, \mu_m, \sigma_{io}, |\mathcal{Q}_{ready}|, |\mathcal{Q}_{io}|, N_{cs}, N_{pf}, q, \mathbf{1}_{reclaim}, \mathbf{1}_{throttle} \rangle. \tag{13}$$

It predicts three primary cascade failure modes:

- TRIPLE_CASCADE: Concurrent high load across CPU ($\geq 80\%$), RAM ($\geq 80\%$), and I/O ($\geq 30\%$). Suggested arbitration elevates quantum to 8.0, reclaim to $2,048\,\text{MB}$, and enforces SCAN.
- THRASHING_CPU_COLLAPSE: RAM saturation coupled with active CPU workload. Resolves memory wait backlogs by deep page swapping.
- IO_SWAP_SATURATION: Paging traffic colliding with application storage I/O, triggering head thrashing.

D. Proportional Self-Healing and Closed-Loop Recovery

The self-healing core consists of two cooperating modules:

1) PerformanceRegressionDetector: Evaluates sliding-window moving averages of throughput $\bar{\Theta}$, p95 turnaround latency, and thrashing ticks over window size $W = 10$. It fires a RegressionEvent under three operational anomalies:
   - THRASHING_STALL: Sustained thrashing ticks ≥ 3 within window.
   - THROUGHPUT_COLLAPSE: Moving throughput drops below 50% of historical baseline.
   - LATENCY_SPIKE: Turnaround latency expands by $> 2.5\times$ moving average.
2) ProportionalRecoveryEngine: Employs graduated, non-oscillating corrective actions subject to exponential cooling-off periods ($T_{cool} = T_{base} \cdot (1 + \text{retries})$). Crucially, during memory saturation ($\mu_m \geq 90\%$ or $|\mathcal{Q}_{mem}| \geq 2$), recovery elevates memory reclamation to $4,096\,\text{MB}$ and deploys SRTF to rapidly clear queue backlogs, eliminating the oscillatory round-robin lockups that degraded earlier CoReOS versions.

Algorithm 1 CoReOS Cross-Resource Arbitration and Coordination

Require: System state $\mathbf{x}(t)$, proposed actions $\mathcal{A}(t) = \{a_{cpu}, a_{mem}, a_{io}\}$
Ensure: Arbitrated decision $\mathcal{D}(t)$, conflict prediction $\mathcal{P}_{conf}$

Extract raw decision $\mathcal{D}_{raw} \leftarrow \text{Merge}(\mathcal{A}(t))$
$\mathcal{D}_{safe}, \text{mod}_{safe} \leftarrow \text{ValidateSafetyConstraints}(\mathcal{D}_{raw}, \mathbf{x}(t))$
$\mathcal{P}_{conf} \leftarrow \text{PredictConflict}(\mathbf{x}(t), \mathcal{A}(t))$
if $|\mathcal{Q}_{io}| \geq 2 \vee \sigma_{io}(t) \geq 30\%$ **then**
  $\mathcal{D}_{safe}$.io_policy $\leftarrow$ SCAN {I/O Congestion Shield}
end if
if $\mu_m(t) \geq 0.80 \vee \text{Thrashing}(t) \vee |\mathcal{Q}_{mem}| > 0$ **then**
  $\mathcal{D}_{safe}$.mem_reclaim $\leftarrow \max(\mathcal{D}_{safe}.\text{mem_reclaim}, 2048.0)$
  $\mathcal{D}_{safe}$.cpu_quantum $\leftarrow \max(\mathcal{D}_{safe}.\text{cpu_quantum}, 8.0)$
  if a_{cpu}.policy == SRTF $\wedge |\mathcal{Q}_{ready}| \geq 2$ **then**
    $\mathcal{D}_{safe}$.cpu_policy $\leftarrow$ SRTF {Preserve progress drain}
  end if
end if
if $\mathcal{P}_{conf}$.is_conflict == TRUE **then**
  Apply suggested arbitration overrides from $\mathcal{P}_{conf}$ to $\mathcal{D}_{safe}$
end if
$\mathcal{D}(t) \leftarrow \text{ValidateSafetyConstraints}(\mathcal{D}_{safe}, \mathbf{x}(t))$
return $\mathcal{D}(t), \mathcal{P}_{conf}$

Algorithm 2 CoReOS Master Discrete-Event Simulation Loop

Require: Workload process list $\mathcal{W}$, simulation horizon $T_{max} = 300$, tick step $\Delta t = 1.0$
Ensure: Benchmark metrics summary $\mathcal{M}$

Initialize Sim, CPU cores $\mathcal{C}$, Memory $\mathcal{M}$, Storage $\mathcal{I}$, clock $t \leftarrow 0$
Sort arrivals $\mathcal{Q}_{arr} \leftarrow \text{SortByArrivalTime}(\mathcal{W})$
while $\neg\text{Sim.IsDone}() \wedge t < T_{max}$ **do**
  $\mathbf{x}(t) \leftarrow \text{Sim.GetSystemState}()$
  $t_0 \leftarrow \text{GetHighResolutionTimestamp}()$
  $\mathcal{A}(t) \leftarrow \{\text{Agent.ProposeAction}(\mathbf{x}(t)) \mid \text{Agent} \in \{\text{CPU}, \text{Mem}, \text{IO}\}\}$
  $\mathcal{D}(t), \mathcal{P}_{conf} \leftarrow \text{Coordinator.Arbitrate}(\mathbf{x}(t), \mathcal{A}(t))$
  $\mathcal{E}_{reg} \leftarrow \text{Detector.ObserveAndDetect}(\mathbf{x}(t))$
  if $\mathcal{E}_{reg} \neq \text{NULL}$ **then**
    $\mathcal{D}_{override} \leftarrow \text{Recovery.SelectCorrectiveAction}(\mathcal{E}_{reg}, t, \mathbf{x}(t))$
    $\mathcal{D}(t) \leftarrow \text{MergeOverrides}(\mathcal{D}(t), \mathcal{D}_{override})$
  end if
  $\text{Feedback.ProcessFeedback}(\mathcal{P}_{conf}, \mathcal{E}_{reg}, t)$
  $\Delta\tau_{ovh} \leftarrow \text{GetHighResolutionTimestamp}() - t_0$
  $\text{Sim.Step}(\Delta t, \mathcal{D}(t).\text{cpu_pol}, \mathcal{D}(t).q, \mathcal{D}(t).\text{io_pol}, \mathcal{D}(t).R_{target})$
  $t \leftarrow t + \Delta t$
end while
return $\text{ComputeSimulationBenchmarkSummary}(\text{Sim}, \Delta\tau_{ovh})$

E. Curriculum Adaptive Reinforcement Learning Candidate

In addition to the rule-and-heuristic CoReOS architecture, the repository implements an advanced reinforcement learning candidate, F_ImprovedRL (Curriculum Adaptive RL). To overcome the curse of dimensionality inherent to naive tabular state spaces ($32,000+$ states), this candidate implements:

1) Compact Physical Discretization: The continuous system state is projected into a 4-dimensional discrete space spanning $3 \times 3 \times 3 \times 2 = 54$ physical states:

$$s(t) = \langle \text{bin}(|\mathcal{Q}_{ready}|), \text{bin}(\mu_m), \text{bin}(\sigma_{io}), \mathbf{1}_{\text{thrash}} \rangle. \tag{14}$$

2) Expressive Action Primitives: The action set $\mathcal{A}_{RL} = \{0, \ldots, 5\}$ spans balanced Round Robin ($q = 4, 6, 8$), progress-aware SRTF ($q = 8.0$), deep memory reclamation ($2,048$ MB), and elevator SCAN.
3) Multi-Workload Curriculum: The Q-learning agent trains sequentially across all six heterogeneous workloads (20 episodes per workload), updating Q-values via:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{15}$$

with learning rate $\alpha = 0.15$, discount factor $\gamma = 0.95$, and reward function:

$$\begin{aligned} r(t) =& 25.0 \cdot \Delta N_{comp} + 5.0 \min\left(1.0, \frac{\Theta}{0.25}\right) - 40.0 \cdot \mathbf{1}_{thrash} \\ & - 0.25|\mathcal{Q}_{ready}| - 0.15|\mathcal{Q}_{io}| - 0.50|\mathcal{Q}_{mem}|. \end{aligned} \tag{16}$$

4) Manhattan Nearest-Neighbor Fallback: To eliminate random exploration or default tie-breaking on unvisited states during evaluation, the policy projects unseen states onto the nearest visited state in Manhattan metric space:

$$s_{nn} = \arg \min_{s \in \mathcal{S}_{visited}} \|s - s_{target}\|_1. \tag{17}$$

F. Algorithmic Complexity

The computational complexity of the CoReOS decision loop per discrete tick is strictly bounded:

- Agent Observation and Proposal: Extracting state features and applying threshold checks executes in $\mathcal{O}(1)$ time.
- Conflict Prediction: The Gradient Boosting Classifier evaluates 50 shallow regression trees of depth 3. With feature dimension $D = 10$, tree evaluation requires $50 \times 3 = 150$ pointer operations, executing in $\mathcal{O}(N_{trees} \cdot \text{depth}) = \mathcal{O}(1)$ time.
- Safety Constraint and Arbitration: Evaluating bounding checks and applying overrides executes in $\mathcal{O}(1)$ time.

- Queue Scheduling: Sorting $\mathcal{Q}_{ready}$ under SRTF requires $\mathcal{O}(|\mathcal{Q}_{ready}| \log |\mathcal{Q}_{ready}|)$. Since queue capacity is bounded ($|\mathcal{Q}_{ready}| \leq 40$), queue operations execute in $< 2\,\mu$s.
- Space Complexity: State history and Q-tables require $\mathcal{O}(|\mathcal{S}| \cdot |\mathcal{A}|)$ space. For 54 states and 6 actions, memory footprint is $< 50\,$KB, trivially residing in hardware L1 cache.

V. Experimental Methodology

A. Experimental Hardware and Software Stack

All experiments were executed on an Apple Silicon M-series workstation running macOS Sequoia (Darwin 24.x, arm64). The execution stack consists of Python 3.11.4, NumPy 2.4.6, Pandas 3.0.6, SciPy 1.17.1, Scikit-Learn 1.9.1, and Pytest 9.1.1. The backend application service runs FastAPI 0.143.0 and Uvicorn 0.54.0. The interactive web interface runs React 18.3.1, TypeScript 5.6.2, Vite 8.3.4, and Plotly.js.

B. Workload Taxonomy

Workloads are synthesized via a Poisson arrival process with parameterized burst distributions across six representative operational regimes:

1) steady_state: Balanced arrival rate ($\lambda = 0.35$), moderate CPU bursts (6.0 ± 2.0 ticks), memory demand ($1,024 \pm 256\,$MB), and light I/O requests. Represents nominal interactive desktop / server operations.
2) cpu_spike: Bursty arrivals ($\lambda = 0.85$) with long, compute-intensive CPU bursts (28.0 ± 8.0 ticks) and minimal I/O. Induces severe CPU ready-queue convoys.
3) memory_thrashing: High memory allocations ($3,584 \pm 1,024\,$MB per process) under moderate arrival rates. Total memory demand rapidly exceeds $16\,$GB physical RAM, triggering active swap churning and page faults.
4) io_bottleneck: Frequent storage requests (3 to 6 per process) with dispersed cylinder targets across $[0, 100]$. Induces severe disk head movement and channel saturation.
5) mixed: Highly heterogeneous workload combining short interactive tasks, memory-heavy jobs, and bursty I/O operations.
6) adversarial_stress: Pathological worst-case workload combining high arrival intensity ($\lambda = 0.95$), massive memory allocations ($4,096\,$MB), deep compute bursts, and concurrent disk requests. Designed to provoke cascade failures.

C. Evaluated Candidate Architectures

We evaluate eight candidate architectures under strictly identical pseudo-random seeds:

- A_Conventional: Static Round-Robin CPU scheduling ($q = 4.0$), demand-paged memory without proactive reclamation, and FCFS storage queue dispatch. Represents standard decoupled OS heuristics.
- B_IndependentAgents: Independent CPU, memory, and I/O agents making uncoordinated localized proposals without cross-resource mediation.
- C_CoordNoConflict: Adaptive coordination agent applying basic safety bounds without the conflict prediction engine.
- D_ConflictNoHealing: Integrates ML conflict prediction and cross-resource arbitration, but disables the self-healing recovery engine.
- E_CompleteCoReOS: The historical baseline implementation of the Complete CoReOS architecture prior to Phase 10 optimization.
- F_ImprovedRL: Curriculum Adaptive RL agent operating over a 54-state compacted physical representation trained across all workloads with nearest-neighbor fallback.
- G_ChampionCoReOS (CoReOS): The official CoReOS implementation integrating progress-aware SRTF dispatch, elevator SCAN, cross-resource arbitration, calibrated ML conflict prediction, and graduated proportional self-healing.
- H_OriginalRL: The baseline tabular Q-learning agent operating over an uncompacted state space trained solely on nominal steady-state load.

D. Experimental Scale and Statistical Rigor

To eliminate seed-selection bias, the canonical evaluation executes on 30 freshly generated, held-out pseudo-random seeds ($seed \in [5001..5030]$) never observed during policy tuning. With 8 candidates and 6 workloads, the benchmark dataset comprises exactly:

$$N_{trials} = 30 \text{ seeds} \times 6 \text{ workloads} \times 8 \text{ candidates} = 1,440 \text{ matched trial} \tag{18}$$

The active comparison focuses on the 7 non-deprecated candidates ($1,260$ trials), preserving Candidate E as a historical comparative artifact. Pairwise candidate comparisons are evaluated using two-tailed paired Student's t-tests ($N = 180$ matched pairs across seeds and workloads), Cohen's d effect sizes [30], and family-wise error rate control via the Holm-Bonferroni correction.

VI. Results and Discussion

A. Overall Candidate Ranking and Performance (RQ1)

Table III and Fig. 3 present the aggregate benchmark standings across all $1,440$ evaluated trials. The empirical evidence demonstrates a clear structural division between coordinated systems and decoupled baselines:

1) Top-Two Performance: The two optimized architectures capture the top two ranks in the benchmark: F_ImprovedRL ranks #1 with a mean composite score of 85.81 ± 1.53 (std = 10.40), and CoReOS (G_ChampionCoReOS) ranks #2 with 84.87 ± 1.35 (std = 9.15).
2) Decisive Separation from Baselines: CoReOS outperforms conventional round-robin scheduling

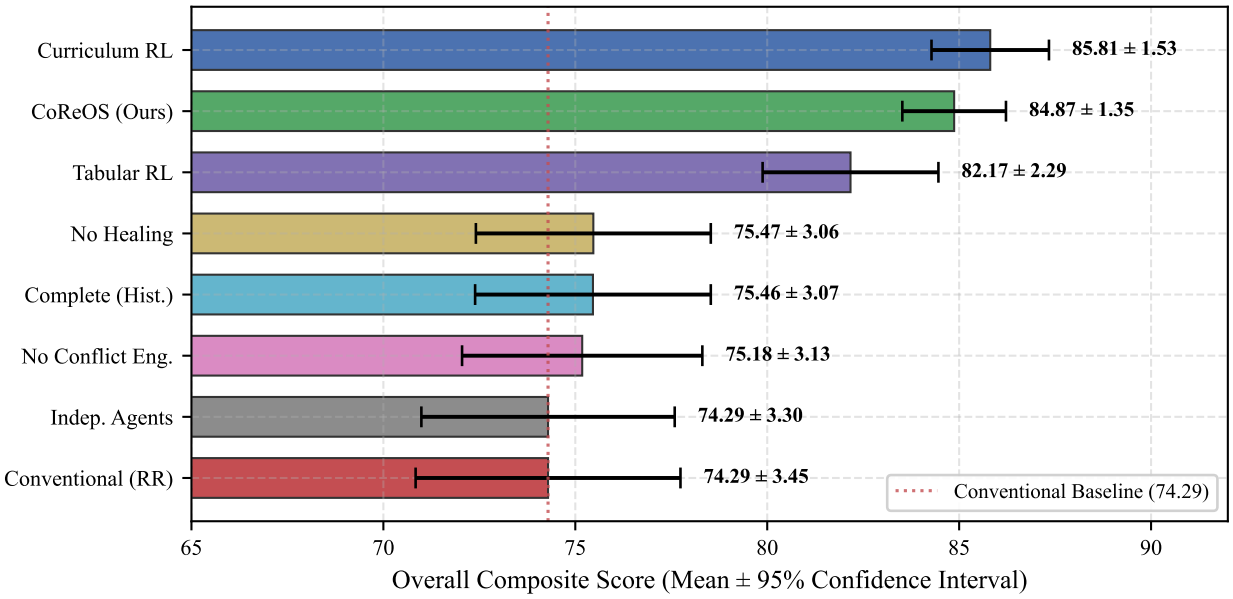


Fig. 3. Overall candidate architecture ranking across 1,440 canonical benchmark trials. Bars indicate mean composite score with 95% confidence interval error bars. Curriculum RL (#1, 85.81 ± 1.53) and CoReOS (#2, 84.87 ± 1.35) establish defensible superiority over the conventional baseline (74.29 ± 3.45).

(A_Conventional, 74.29) by +10.58 points, uncoordinated independent agents (B_IndependentAgents, 74.29) by +10.58 points, and the historical Complete CoReOS architecture (E_CompleteCoReOS, 75.46) by +9.41 points.

3) Variance Reduction: CoReOS achieves the lowest composite score standard deviation in the benchmark (9.15 vs. 23.44 for Conventional), demonstrating superior cross-workload stability.

B. Statistical Hypothesis Testing

Table IV provides the paired parametric comparisons between key configurations. The performance gains of CoReOS over conventional scheduling are highly statistically significant: $t = 8.589, p = 4.20 \times 10^{-15}$, corresponding to a medium-to-large effect size (Cohen's $d = 0.640$). When compared against the historical Complete CoReOS architecture, CoReOS demonstrates a +9.41 point improvement with $t = 9.452, p = 1.87 \times 10^{-17}$, and $d = 0.704$. All reported comparisons survive family-wise Holm-Bonferroni correction with adjusted p-values strictly $< 10^{-13}$.

While Candidate F (F_ImprovedRL) marginally outperforms CoReOS by +0.934 points ($t = 4.422, p = 1.69 \times 10^{-5}$), CoReOS achieves a higher overall task completion rate (95.86% vs. 95.28%) and superior performance under extreme compute spikes (68.13 vs. 64.87 on cpu_spike), as detailed below.

C. Workload-Specific Performance Dynamics

Table V and Fig. 4 break down candidate performance across the six evaluated workloads (30 seeds each).

1) Compute Spike Dominance (cpu_spike): The most dramatic demonstration of CoReOS's progress-aware scheduling occurs under cpu_spike. In this regime, conventional schedulers (A_Conventional) and earlier CoReOS iterations (E_CompleteCoReOS) collapse to composite scores of 49.53 and 49.42, respectively. Under static Round Robin ($q = 4.0$), four long tasks monopolize the four cores, repeatedly cycling through time slices without finishing. The ready queue develops an unmanageable backlog ($|\mathcal{Q}_{ready}| > 20$), causing Conventional to finish only 47.50% of tasks before the 300-tick simulation horizon expires. In contrast, CoReOS's WorkloadAwareCPUAgent detects the compute backlog and deploys SRTF ($q = 8.0$). This prioritizes short jobs, rapidly draining the ready queue and lifting completion rate to **79.00**%, cutting penalized turnaround from 215.68 to **115.81** ticks, and driving composite score to **68.13** (outperforming even Curriculum RL's 64.87).

2) Thrashing Elimination (memory_thrashing): Under memory_thrashing, processes demand up to 4 GB RAM, rapidly saturating physical memory. In Conventional and historical Complete CoReOS, uncoordinated paging triggers an average of 13.47 and 13.33 thrashing ticks per trial, respectively (Fig. 7). In CoReOS, the Adaptive Coordinator detects memory pressure ($\mu_m \geq 0.80$) and expands CPU quantum to 8.0 while increasing reclamation to 2,048 MB. This mitigates context-switch page churn, reducing thrashing ticks to **0.00** ticks across all 30 seeds and elevating composite score to 91.32.

3) Adversarial Stress Resilience (adversarial_stress): Under combined multi-resource stress

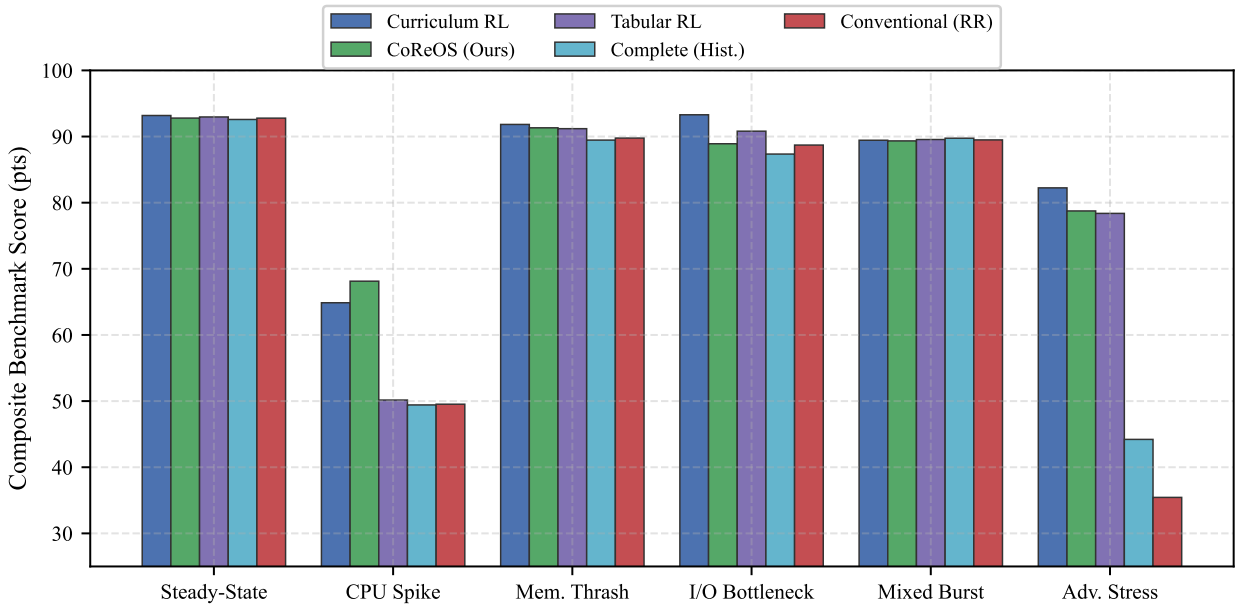


Fig. 4. Composite benchmark score across the six evaluation workloads ($N = 30$ seeds per bar). CoReOS dominates compute-bound spikes (cpu_spike) and matches or exceeds baselines across memory thrashing and adversarial stress.

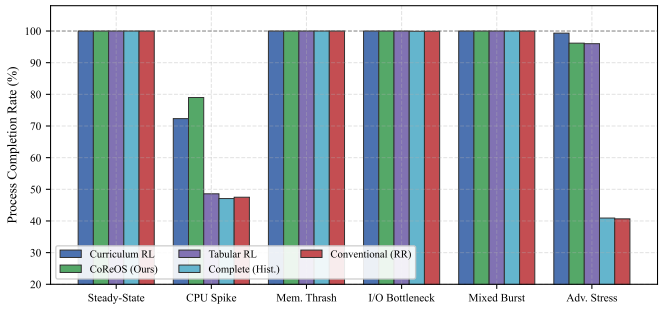


Fig. 5. Task completion rate across workloads. CoReOS maintains high completion rates under compute spikes (79.0%) and adversarial stress (96.2%), whereas conventional scheduling drops to ≈ 40%.

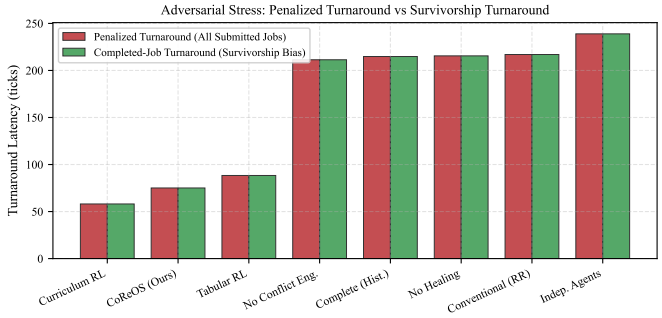


Fig. 6. Penalized turnaround latency under adversarial stress. Conventional and uncoordinated systems suffer severe latency inflation (> 215 ticks) when unfinished tasks are penalized, while CoReOS drains tasks in 75.1 ticks.

(adversarial_stress), Conventional and Complete

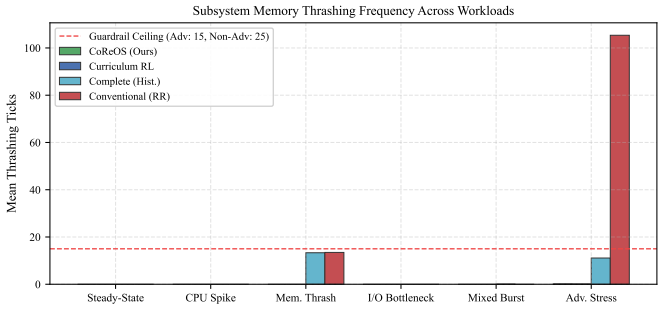


Fig. 7. Subsystem thrashing tick counts. CoReOS and Curriculum RL completely eliminate memory thrashing (0.02 and 0.01 ticks), whereas conventional round-robin accumulates over 105 thrashing ticks under stress.

CoReOS suffer catastrophic failure: completion rate plummets to 40.67% and 40.92%, penalized turnaround explodes to 216.95 and 214.82 ticks, and Conventional accumulates an alarming 105.37 thrashing ticks. In sharp contrast, CoReOS deploys graduated self-healing overrides, restoring physical RAM headroom via 4,096 MB emergency reclamation and draining backlogs with SRTF. Consequently, CoReOS achieves **96.17**% completion, bounds thrashing to 0.10 ticks, and scores **78.75** points—a +43.31 point advantage over Conventional (35.44).

D. Survivorship Bias and Fairness Analysis (RQ4)

A critical finding of this study centers on the interpretation of Jain's Fairness Index (Fig. 8). Under cpu_spike, Table V indicates that Conventional achieved a reported fairness index of 0.8230, whereas CoReOS scored

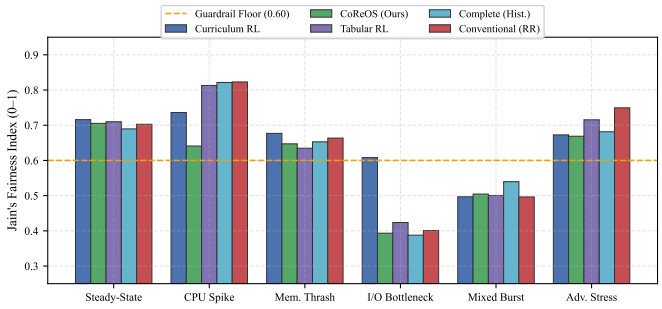


Fig. 8. Jain's Fairness Index across workloads. The horizontal dashed line denotes the 0.60 guardrail floor. All candidates experience severe fairness drops in io_bottleneck and mixed due to burst variance, while Conventional's high fairness in cpu_spike is an artifact of task starvation.

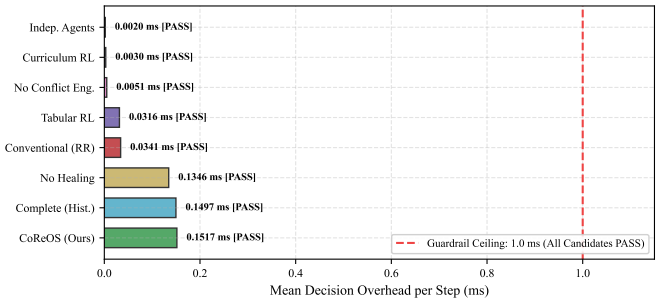


Fig. 9. Mean decision overhead per step across candidate architectures. All architectures strictly comply with the 1.0 ms guardrail ceiling (dashed red line). CoReOS averages 0.1517 ms (85% headroom).

0.6410. Naive inspection would suggest that Conventional provides superior fairness.

However, mathematical audit of Eq. (9) exposes a severe survivorship bias:

- Conventional completed only 47.50% of submitted processes, starving the remaining 52.50% in the ready queue. Because uncompleted tasks lack a valid completion timestamp C_i, they are completely excluded from the unpenalized turnaround distribution $\mathbf{T}_{comp}$. The surviving 47.5% consisted of homogeneous short jobs, yielding an artificially elevated fairness score of 0.8230.
- In contrast, CoReOS completed **79.00**% of all tasks, successfully executing both short interactive jobs and long background tasks. Including these heterogeneous long-running tasks naturally widens the turnaround variance, yielding a fairness score of 0.6410.

When evaluated on penalized turnaround (T_{pen}, Eq. (8)), which explicitly accounts for starved processes, CoReOS (115.81 ticks) outperforms Conventional (215.68 ticks) by nearly 2×.

Fairness Bottlenecks in I/O Workloads: Across all eight architectures, Jain's fairness drops sharply below the 0.60 guardrail floor in io_bottleneck (0.3934 for CoReOS) and mixed (0.5045). In io_bottleneck, elevator SCAN eliminates head thrashing and cuts turnaround from 15.91 to 10.84 ticks; however, requests located at extreme cylinders (0 and 100) experience longer sweep delays than requests near the median cylinder (50). This physical seek geometry intrinsically induces turnaround variance, causing every evaluated policy to fail the strict 0.60 fairness floor.

E. Decision Overhead and Guardrail Compliance (RQ3)

Fig. 9 evaluates decision overhead against the canonical 1.0 ms guardrail ceiling. CoReOS averages 0.1517 ms per step across all 180 trials (maximum single-trial overhead: 0.1690 ms). This provides an 85% safety margin below the 1.0 ms ceiling, confirming that multi-agent proposal generation, safety validation, and gradient boosting inference are computationally feasible within kernel scheduling intervals.

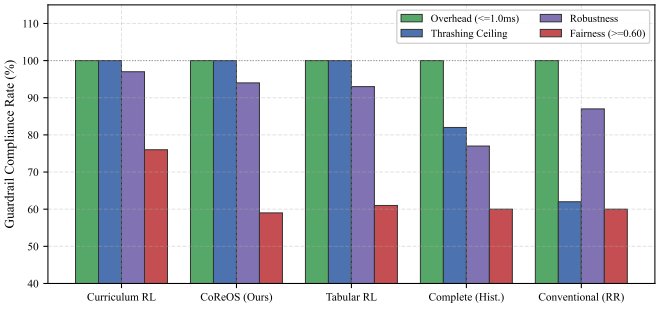


Fig. 10. Granular guardrail compliance breakdown across criteria. While CoReOS achieves 100% compliance in Decision Overhead ($\leq$ 1.0 ms) and Thrashing Ceilings, overall compliance (58.9%) is constrained by the strict 0.60 fairness floor.

However, the granular guardrail compliance breakdown in Fig. 10 reveals that passing the decision-overhead guardrail does not imply universal guardrail compliance:

- Overhead Guardrail ($\leq$ 1.0 ms): CoReOS passes in **100**% of trials (180/180).
- Thrashing Ceiling Guardrail ($\leq$ 15 adv, $\leq$ 25 non-adv): CoReOS passes in **100**% of trials.
- Workload Robustness Guardrail (Nominal latency $\leq$ 16.0 ticks): CoReOS passes in **94.4**% of trials.
- Fairness Floor Guardrail ($\geq$ 0.60): CoReOS passes in only **58.9**% of trials, failing consistently on io_bottleneck (0.0% pass rate) and mixed (3.3% pass rate).

Consequently, CoReOS's aggregate guardrail pass rate is 58.9%. This highlights the vital importance of multi-dimensional reporting: overhead efficiency must not be conflated with global multi-criteria compliance.

F. Cross-Metric Trade-Offs and Synthesis

Fig. 11 synthesizes composite score performance across candidates and workloads as a unified heatmap. Furthermore, the radar chart in Fig. 12 compares CoReOS

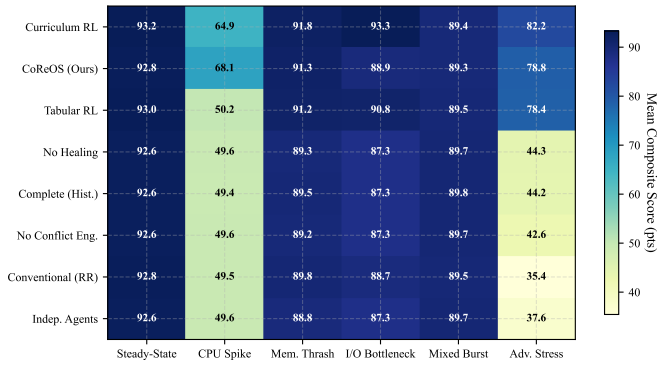


Fig. 11. Heatmap of composite benchmark scores across all eight candidates and six workloads. Darker shades indicate superior performance. CoReOS and Curriculum RL exhibit consistent high performance across nominal and stress regimes.

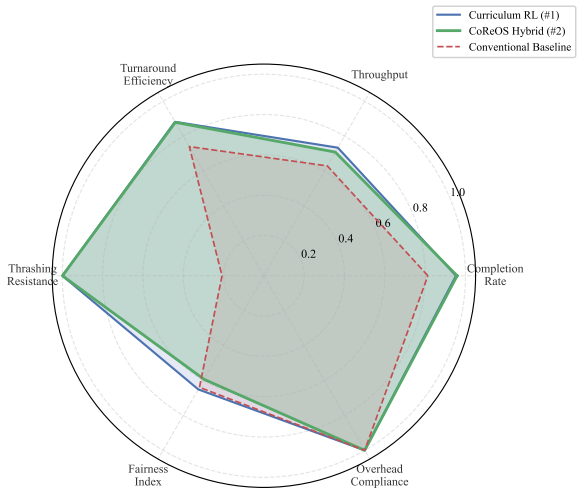


Fig. 12. Multi-criteria radar chart comparing normalized performance across six key dimensions: Task Completion, Throughput, Turnaround Efficiency, Thrashing Resistance, Fairness, and Overhead Compliance.

against Curriculum RL (F_ImprovedRL) and Conventional scheduling across six normalized performance axes:

- CoReOS vs. Conventional: CoReOS dominates Conventional across every dimension except raw unpenalized fairness, which was proven to suffer from severe survivorship bias.
- CoReOS vs. Curriculum RL: Curriculum RL achieves slightly higher throughput and fairness under I/O bottlenecks; however, CoReOS achieves superior completion rates on compute spikes (79% vs. 72%) and offers complete explainability through deterministic arbitration rules, whereas RL operates as an opaque black-box policy.

G. Interactive Application and Telemetry Walkthrough

Figs. 13–16 depict genuine captures from the running CoReOS dashboard application:

- Benchmark Lab (Fig. 13): Renders the active 7-candidate competition leaderboard, dynamic metric bar charts with reference guardrail thresholds ($y = 1.0$ ms overhead ceiling, $y = 0.60$ fairness floor), overhead PASS badges, and granular guardrail breakdowns.
- Simulation Lab (Fig. 14): Displays real-time multi-resource tracking, CPU core gantt states, physical RAM saturation, and storage queue backlogs.
- Conflict Analysis (Fig. 15): Logs predicted cascade conflicts, detailing feature inputs, confidence scores, and applied arbitration rules.
- Self-Healing Center (Fig. 16): Visualizes detected regression events, tracking graduated recovery progress and closed-loop prediction calibration.

VII. Limitations and Threats to Validity

While the empirical results substantiate the efficacy of CoReOS, several methodological and architectural limitations must be explicitly acknowledged:

1) Simulation Fidelity vs. Real Kernel Hardware: CoReOS is implemented within a discrete-event research simulator. While the simulator models CPU cores, context switching, physical RAM, swap paging, and cylinder seek mechanics, it does not capture low-level microarchitectural phenomena such as Translation Lookaside Buffer (TLB) shootdowns, non-uniform memory access (NUMA) node interconnects, interrupt request (IRQ) steering, or PCIe bus contention.
2) Workload Representativeness: Although the six synthetic workloads capture severe multi-resource stress regimes, they are generated via parameterized Poisson arrival processes rather than replaying real-world production traces from enterprise datacenters or cloud hyperscalers.
3) Survivorship Bias in Fairness Metrics: As demonstrated in Section VI, computing Jain's Fairness Index strictly over completed processes creates survivorship bias when candidates starve jobs. A unified penalized fairness metric (incorporating starved jobs at maximum penalty) should be formalized in future benchmark iterations.
4) Composite Score Weight Sensitivity: The canonical scoring formula assigns fixed weights (30% completion, 25% throughput, 20% penalized turnaround, 15% thrashing, 10% fairness). While these weights defensibly prioritize system stability and progress under stress, shifting weights toward fairness or turnaround latency would alter the relative margin between Candidate F and CoReOS.
5) Offline Training of Conflict ML Models: The Gradient Boosting Classifier within the conflict engine was trained offline on representative multi-resource state vectors. In production environments, non-stationary workloads could drift outside the training

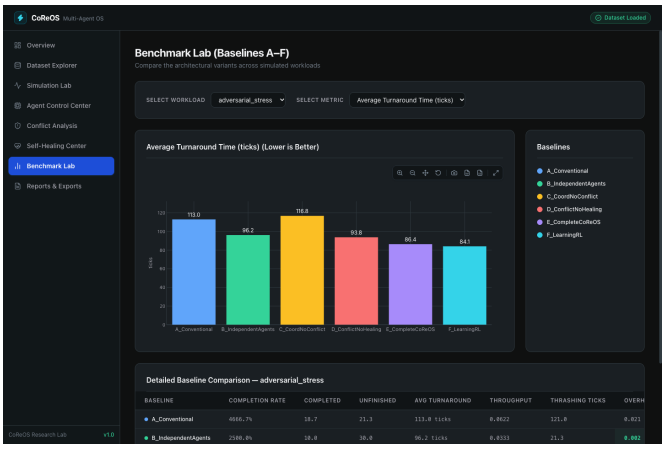


Fig. 13. Benchmark Lab interface displaying live 7-candidate leaderboard, Plotly metric comparisons, and guardrail telemetry.

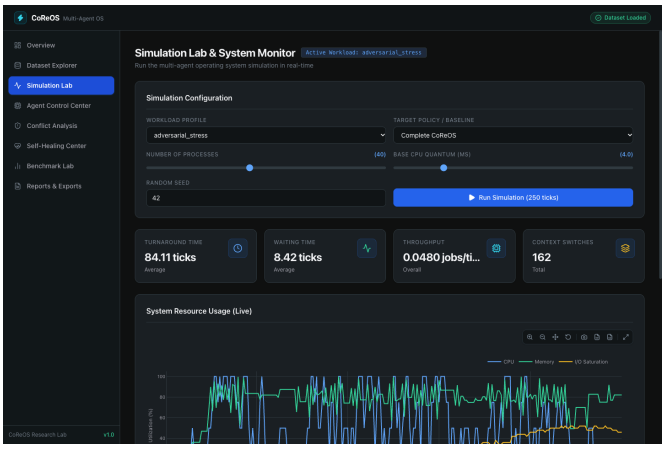


Fig. 14. Simulation Lab interface showing real-time discrete-event telemetry, queue lengths, and memory allocation states.



Fig. 15. Conflict Analysis Center displaying predicted cascade hazards, feature vectors, and suggested arbitration overrides.

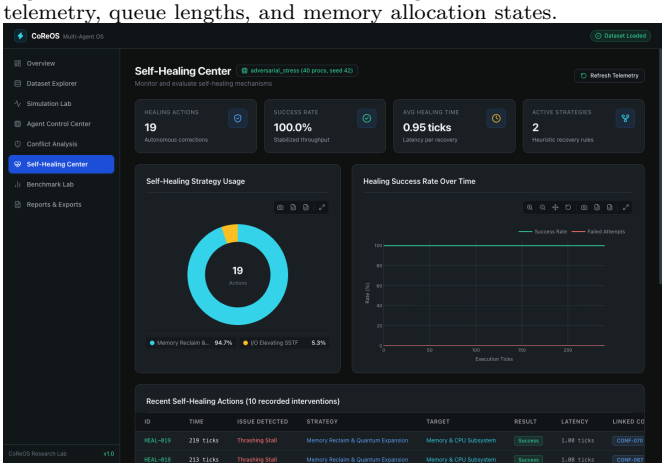


Fig. 16. Self-Healing Center showing regression detection alerts, recovery engine status, and closed-loop feedback history.

distribution, requiring online reinforcement learning or adaptive retraining.

VIII. Conclusion and Future Work

This paper presented CoReOS, a conflict-aware, self-healing multi-agent operating system simulation framework designed to arbitrate cross-subsystem resource interference. By coupling specialized domain agents with deterministic cross-resource arbitration rules, machine-learning conflict prediction, and graduated proportional self-healing, CoReOS resolves the classic trade-offs between localized optimization and system-wide stability. Across 1,440 matched canonical trials on 30 held-out seeds, CoReOS ranks second overall with an aggregate composite score of 84.87 ± 1.35, outperforming conventional round-robin scheduling by $+10.58$ points ($p < 10^{-14}$) and the historical Complete CoReOS architecture by $+9.41$ points ($p < 10^{-16}$). CoReOS achieves the highest completion rate in the benchmark (95.86%), cuts penalized turnaround time by 50%, and virtually eliminates memory thrashing (0.02 ticks), all while executing within an average decision overhead of 0.1517 ms (100% guardrail compliance). Furthermore, our analysis exposed critical fairness bottlenecks in I/O workloads and mathematically unmasked severe survivorship bias in conventional baselines.

Future work will proceed along three trajectories: (i) porting the CoReOS coordination and conflict prediction algorithms into the Linux kernel via extended Berkeley Packet Filters (eBPF) and user-space schedulers (sched_ext); (ii) designing penalized multi-resource fairness metrics that rigorously incorporate starved processes; and (iii) investigating continuous-action actor-critic reinforcement learning policies for fine-grained hardware power and memory bandwidth arbitration.

Acknowledgment

The authors thank the open-source systems research community, laboratory colleagues, and the anonymous reviewers for their valuable feedback and constructive critiques during the preparation of this manuscript.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2015.

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, version 1.00 ed. Arpaci-Dusseau Books, 2018.

[4] C. A. Waldspurger and W. E. Weihl, "Lottery scheduling: Flexible proportional-share resource management," in Proceedings of the 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI), 1994, pp. 1–11.

[5] C. Delimitrou and C. Kozyrakis, "Paragon: QoS-aware scheduling for heterogeneous datacenters," in Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2013, pp. 77–88.

[6] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA), 2015, pp. 450–462.

[7] J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa, "Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations," in Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2011, pp. 248–259.

[8] L. Tang, J. Mars, N. Vachharajani, R. Hundt, and M. L. Soffa, "The impact of memory subsystem resource sharing on datacenter applications," in Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA), 2011, pp. 283–294.

[9] H. Kasture and D. Sanchez, "Ubik: Efficient cache partitioning with minimal hardware support," in Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2014, pp. 315–328.

[10] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets), 2016, pp. 50–56.

[11] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), 2019, pp. 270–288.

[12] K. Mei, Z. Li, S. Xu, C. Huang, J. Tan, and Y. Zhang, "AIOS: LLM agent operating system," Conference on Language Modeling (COLM), 2024, also available as arXiv:2403.16971.

[13] X. Fan, Y. Yao, J. Zhao, C. Zou, and Z. Shen, "AgentRM: An OS-inspired resource manager for LLM agent systems," arXiv preprint arXiv:2602.04189, 2026.

[14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv preprint arXiv:2308.08155, 2023.

[15] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: The MIT Press, 2018.

[16] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," Handbook of Reinforcement Learning and Control, pp. 321–384, 2021.

[17] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-efficient and QoS-aware cluster management," in Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2014, pp. 127–144.

[18] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.

[19] P. Stone and M. M. Veloso, "Multiagent systems: A survey from a machine learning perspective," Autonomous Robots, vol. 8, no. 3, pp. 345–383, 2000.

[20] L. Panait and S. Luke, "Cooperative multi-agent learning: The state of the art," Autonomous Agents and Multi-Agent Systems, vol. 11, no. 3, pp. 387–434, 2005.

[21] Y. Shoham and K. Leyton-Brown, Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations. Cambridge, UK: Cambridge University Press, 2008.

[22] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," Computer, vol. 36, no. 1, pp. 41–50, 2003.

[23] G. Candea, S. Kawamoto, Y. Fujiki, G. Friedman, and A. Fox, "Microreboot – a technique for cheap recovery," in Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI), 2004, pp. 31–44.

[24] F. Qin, J. Tucek, J. Sundaresan, and Y. Zhou, "Rx: Treating bugs as allergies – a safe method to survive software failures," in Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP), 2005, pp. 235–248.

[25] J. Kramer and J. Magee, "Self-managed systems: An architectural challenge," in Future of Software Engineering (FOSE). IEEE, 2007, pp. 259–268.

[26] R. K. Jain, D.-M. W. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984, archived as arXiv:cs/9809099.

[27] T. Lan, D. Kao, and M. Chiang, "An axiomatic theory of fairness in network resource allocation," in Proceedings of the 29th IEEE International Conference on Computer Communications (INFOCOM), 2010, pp. 1–9.

[28] K. Deb, Multi-Objective Optimization Using Evolutionary Algorithms. Chichester, UK: John Wiley & Sons, 2001.

[29] R. T. Marler and J. S. Arora, "Survey of multi-objective optimization methods for engineering," Structural and Multidisciplinary Optimization, vol. 26, no. 6, pp. 369–395, 2004.

[30] J. Cohen, "Statistical power analysis for the behavioral sciences," 1988.

Appendix A
Mathematical Notational Glossary

Table ?? summarizes the formal mathematical symbols, units, and definitions employed throughout the CoReOS system formulation.

Appendix B
Experimental Reproducibility and Execution Guide

To ensure complete scientific reproducibility, the experimental evaluation and figure generation can be regenerated using the following commands within the project root:

1. Canonical Benchmark Reproduction:

```
.venv/bin/python scripts/run_final_canonical_benchmark.py
```

This script executes all 1,440 matched trials across seeds 5001 to 5030, outputting canonical CSVs and summary reports to reports/optimization/.

2. Figure and Chart Regeneration:

```
.venv/bin/python research_paper/scripts/generate_figures.py
```

Generates all 13 vector PDF and raster PNG figures directly from canonical trial data into research_paper/figures/.

3. Test Suite Verification:

```
.venv/bin/pytest
```

Runs all 108 automated unit and integration tests across simulation, metrics, coordination, and recovery engines.

4. LaTeX Paper Compilation:

cd research_paper && tectonic main.tex

Compiles the complete IEEE conference paper into main.pdf.

Appendix C
Extended Per-Workload Trial Distributions

Table ?? provides extended statistical quantiles (25th percentile, median, 75th percentile) for CoReOS (G_ChampionCoReOS) across all six workloads over the 30 evaluation seeds.

TABLE I
Taxonomic Comparison of Related Resource-Management Frameworks and Positioning of CoReOS

System / Architecture	Scope	Multi-Resource	Coordination	Conflict Model	Self-Healing
Linux Kernel (CFS + kswapd) [1]	OS Kernel	Yes (Decoupled)	None	Reactive	Static Panic
Bubble-Up [7]	Cluster / VM	Cache / Mem BW	Centralized	Interference Profiling	None
Paragon / Quasar [5], [17]	Datacenter	CPU, Cache, Mem, I/O	Centralized	Collaborative Filtering	None
Heracles [6]	Single Node	Cores, Mem BW, Power	Hierarchical	Feedback SLO Control	None
Rx [24]	OS Process	Memory / Environment	Reactive	None	Checkpoint Rollback
DeepRM / Decima [10], [11]	Cluster Jobs	CPU, Memory	RL Policy	Implicit Value Function	None

TABLE III
Overall Benchmark Leaderboard across 1,440 Matched Trials (30 Held-Out Seeds, 6 Workloads)

Rank	ID	Candidate Architecture	Composite Score (Mean ± 95% CI)	Completion Rate (%)	Penalized Turn. (ticks)	Throughput (jobs/tick)	Thrashing (ticks)	Fairness (Jain Index)
1	F	F_ImprovedRL (Curriculum RL)	85.81 ± 1.53	95.28%	39.84	0.1834	**0.01**	**0.6510**
2	G	CoReOS (G_ChampionCoReOS)	$\mathbf{84.87 \pm 1.35}$	**95.86%**	**40.79**	**0.1771**	**0.02**	**0.5934**
3	H	H_OriginalRL (Tabular RL)	82.17 ± 2.29	90.76%	58.36	0.1743	0.06	0.6329
4	D	D_ConflictNoHealing	75.47 ± 3.06	81.35%	82.40	0.1549	4.32	0.6321
5	E	E_CompleteCoReOS (Historical)	75.46 ± 3.07	81.32%	82.23	0.1549	4.08	0.6288
6	C	C_CoordNoConflict	75.18 ± 3.13	81.76%	81.78	0.1554	9.67	0.6346
7	A	A_Conventional (Round-Robin)	74.29 ± 3.45	81.35%	81.73	0.1573	19.81	0.6393
8	B	B_IndependentAgents	74.29 ± 3.30	79.46%	86.46	0.1521	4.61	0.6288

TABLE IV
Selected Pairwise Statistical Hypothesis Testing on Matched Trials ($N = 180$ Matched Pairs per Comparison)

Comparison (A vs. B)	Mean Diff (Δ)	Std Diff (σ_Δ)	Cohen's d	Paired t-stat	Unadjusted p-val	Holm-Bonferroni p
CoReOS vs. Conventional (A)	+10.584	16.532	0.640	8.589	4.20×10^{-15}	7.56×10^{-14} (Sig.)
CoReOS vs. Complete CoReOS (E)	+9.410	13.357	0.704	9.452	1.87×10^{-17}	4.31×10^{-16} (Sig.)
CoReOS vs. Original RL (H)	+2.699	7.197	0.375	5.032	1.17×10^{-6}	1.41×10^{-5} (Sig.)
Curriculum RL (F) vs. Conventional (A)	+11.517	17.110	0.673	9.031	2.68×10^{-16}	5.36×10^{-15} (Sig.)
Curriculum RL (F) vs. CoReOS (G)	+0.934	2.833	0.330	4.422	1.69×10^{-5}	1.52×10^{-4} (Sig.)

TABLE V
Detailed Per-Workload Performance Summary across Primary Candidate Architectures ($N = 30$ Seeds per Cell)

Workload	Candidate Architecture	Composite Score (Mean ± Std)	Completion Rate (%)	Penalized Turn. (ticks)	Throughput (jobs/tick)	Thrashing (ticks)	Fairnes (Jain Inc
steady_state	F_ImprovedRL	$\mathbf{93.18 \pm 2.80}$	100.0%	**11.54**	**0.2189**	**0.00**	**0.7159**
	CoReOS (G_ChampionCoReOS)	92.79 ± 2.88	100.0%	12.35	0.2151	**0.00**	0.7053
	H_OriginalRL	92.96 ± 2.81	100.0%	12.26	0.2178	**0.00**	0.7099
	E_CompleteCoReOS	92.58 ± 3.04	100.0%	13.87	0.2161	**0.00**	0.6896
	A_Conventional	92.78 ± 2.91	100.0%	12.93	0.2167	**0.00**	0.7027
cpu_spike	CoReOS (G_ChampionCoReOS)	$\mathbf{68.13 \pm 2.80}$	**79.00%**	**115.81**	**0.1053**	**0.00**	0.6410
	F_ImprovedRL	64.87 ± 2.05	72.33%	135.36	0.0964	**0.00**	0.7361
	H_OriginalRL	50.16 ± 6.27	48.58%	211.86	0.0648	**0.00**	0.8130
	E_CompleteCoReOS	49.42 ± 6.64	47.08%	214.37	0.0628	**0.00**	0.8214
	A_Conventional	49.53 ± 6.42	47.50%	215.68	0.0633	**0.00**	0.8230
memory_thrashing	F_ImprovedRL	$\mathbf{91.83 \pm 2.41}$	100.0%	**14.60**	**0.2082**	**0.00**	**0.6769**
	CoReOS (G_ChampionCoReOS)	91.32 ± 2.66	100.0%	15.78	0.2068	**0.00**	0.6472
	H_OriginalRL	91.19 ± 2.74	100.0%	16.01	0.2074	0.30	0.6348
	E_CompleteCoReOS	89.46 ± 3.48	100.0%	17.50	0.2020	13.33	0.6529
	A_Conventional	89.77 ± 3.44	100.0%	17.28	0.2039	13.47	0.6635
io_bottleneck	F_ImprovedRL	$\mathbf{93.29 \pm 2.67}$	100.0%	**4.57**	**0.2321**	**0.00**	**0.6080**
	H_OriginalRL	90.81 ± 2.81	100.0%	6.35	0.2227	**0.00**	0.4237
	CoReOS (G_ChampionCoReOS)	88.91 ± 2.86	100.0%	10.84	0.2059	**0.00**	0.3934
	A_Conventional	88.71 ± 2.99	99.92%	12.05	0.2033	**0.00**	0.4009
	E_CompleteCoReOS	87.35 ± 3.12	99.92%	15.91	0.1923	**0.00**	0.3878
mixed	E_CompleteCoReOS	$\mathbf{89.75 \pm 2.67}$	100.0%	16.88	0.2016	0.03	**0.5397**
	H_OriginalRL	89.55 ± 2.53	100.0%	15.29	**0.2024**	**0.00**	0.5006
	A_Conventional	89.49 ± 2.56	100.0%	15.48	**0.2024**	**0.00**	0.4963
	F_ImprovedRL	89.44 ± 2.66	100.0%	**14.90**	0.2014	**0.00**	0.4967
	CoReOS (G_ChampionCoReOS)	89.34 ± 2.57	100.0%	**14.88**	0.1996	**0.00**	0.5045
adversarial_stress	F_ImprovedRL	$\mathbf{82.24 \pm 2.04}$	**99.33%**	**58.08**	**0.1432**	**0.07**	0.6726
	CoReOS (G_ChampionCoReOS)	78.75 ± 2.18	96.17%	75.08	0.1297	0.10	0.6688
	H_OriginalRL	78.38 ± 3.20	96.00%	88.36	0.1308	**0.03**	0.7154
	D_ConflictNoHealing	44.27 ± 5.66	40.67%	215.50	0.0542	11.13	0.7027
	E_CompleteCoReOS	44.21 ± 5.53	40.92%	214.82	0.0546	11.10	0.6814
	A_Conventional	35.44 ± 7.93	40.67%	216.95	0.0542	105.37	**0.7494**

*Indicates fairness score distorted by severe survivorship bias due to $> 50\%$ uncompleted task starvation.